Constraint Satisfaction Problems Study Notes

Part 4: Heuristics for Better Backtracking

Learning Goal

By the end of this part, you should understand how heuristics improve backtracking search by making better choices about **which variable to assign next** and **which value to try first**. You should also understand the ideas behind MRV, degree heuristic, and least-constraining value, and why these heuristics reduce wasted search.

1. Core Idea

Basic backtracking is systematic, but it is still inefficient if it chooses variables and values blindly. Heuristics improve the search by guiding it toward decisions that are more informative and less likely to waste time.

Instead of asking only “Can I assign something next?”, heuristics ask:

- Which variable is most urgent to assign?
- Which value is least likely to cause trouble later?

This does not change the correctness of backtracking. It changes the **order** in which search is performed, which can make an enormous difference in efficiency.

> **Things to Note:**
>
> - Heuristics do not change what counts as a solution.
> - They change the order in which possibilities are explored.
> - A good ordering can make failure appear early and success appear sooner.

2. Why Heuristics Matter

In a CSP, some choices are much more important than others. If you delay an important difficult variable, you may spend a long time exploring branches that were doomed from the start. If you assign that difficult variable early, failure can be detected sooner.

Similarly, some values leave many future options open, while others block the remaining search. If you choose a restrictive value too early, the branch may collapse later.

Heuristics try to exploit this idea.

Main Strategy:

Choose variables that expose difficulty early, and choose values that preserve flexibility for later.

3. Variable-Ordering Heuristic: Minimum Remaining Values (MRV)

The **Minimum Remaining Values** heuristic chooses the variable with the fewest legal values left in its domain.

This is often called the **most constrained variable** heuristic.

Why MRV makes sense

If a variable has very few remaining choices, it is the most likely place where failure will happen. So it is usually smart to assign it first.

This is known as the **fail-first principle**:

Try to discover unavoidable failure as early as possible.

Example idea

Suppose in exam scheduling:

- `CS101` has 3 possible slots left
- `CS102` has 2 possible slots left
- `MATH201` has 1 possible slot left
- `ENG101` has 3 possible slots left

Then MRV says to assign `MATH201` first, because it has the smallest domain left.

Variable	Remaining Legal Values	MRV Priority
`CS101`	3	Lower
`CS102`	2	Medium
`MATH201`	1	Highest
`ENG101`	3	Lower

High-Yield Point:

MRV tries to detect dead ends early. It does not look for the easiest variable. It looks for the most constrained one.

Common Confusion:

> Students often think MRV means “pick the variable with the smallest original domain.” That is not precise enough. MRV is about the **remaining legal values at the current stage of search**, not merely the original domain size.

4. Tie-Breaking Heuristic: Degree Heuristic

Sometimes two or more variables tie under MRV. In that case, we need another rule to break the tie.

The **degree heuristic** chooses the variable that is involved in the largest number of constraints on other unassigned variables.

Why degree heuristic makes sense

If a variable interacts with many others, assigning it now has a strong effect on the remaining problem. So if there is a tie, it is useful to choose the variable that constrains the rest of the search the most.

Example idea

Suppose two exams each have two remaining slots. If one of them conflicts with three other unassigned exams while the other conflicts with only one, then the degree heuristic prefers the first one.

Variable	Remaining Values	Number of Constraints on Unassigned Variables	Chosen by Degree?
`CS102`	2	3	Yes
`ENG101`	2	1	No

> **Simple Rule:**
>
> MRV asks, “Which variable is most constrained right now?” Degree asks, “Which variable constrains the rest the most?”

5. Value-Ordering Heuristic: Least-Constraining Value (LCV)

Once a variable has been chosen, the next question is: which value should be tried first?

The **Least-Constraining Value** heuristic chooses the value that rules out the fewest values for neighboring unassigned variables.

Why LCV makes sense

A good value is one that leaves the future as open as possible. If one value creates many future conflicts while another leaves lots of options, try the flexible one first.

Example idea

Suppose in map coloring we are assigning `SA`. If coloring `SA = red` would force several neighbors into tight restrictions, while `SA = blue` still leaves them many choices, then LCV prefers `blue`.

Exam scheduling intuition

If assigning `CS101 = Mon-AM` would block that slot for several other exams, while assigning `CS101 = Tue-AM` affects fewer remaining possibilities, then LCV prefers `Tue-AM`.

> **Things to Note:**
>
> - MRV and degree choose the variable.
> - LCV chooses the value.
> - LCV is about preserving future flexibility.

6. Putting the Heuristics Together

These heuristics are usually combined in the following way:

1. Use **MRV** to choose the next variable.
2. If there is a tie, use the **degree heuristic**.
3. Once the variable is chosen, use **LCV** to order its possible values.

This gives a far smarter version of backtracking than simply assigning variables left-to-right and trying values in arbitrary order.

Heuristic	Used For	Main Goal
MRV	Select next variable	Find dead ends early
Degree heuristic	Break ties between variables	Choose variable that affects the rest most
LCV	Order values	Preserve future options

7. Worked Understanding Using Exam Scheduling

Suppose the current partial assignment has already reduced the domains as follows:

- `CS101: {Mon-AM, Tue-AM}`
- `CS102: {Mon-PM}`
- `MATH201: {Mon-PM, Tue-AM}`
- `ENG101: {Mon-PM, Tue-AM}`

Applying MRV

`CS102` has only one remaining value, so MRV chooses `CS102` first.

After assigning CS102

Suppose now there is a tie between `MATH201` and `ENG101`, both with two legal values.

If `MATH201` is connected by constraints to more unassigned variables, then the degree heuristic chooses `MATH201`.

Applying LCV

Now assume `MATH201` can be assigned either `Mon-PM` or `Tue-AM`. If assigning `Mon-PM` would eliminate options for other exams, while `Tue-AM` leaves more choices open, LCV prefers `Tue-AM`.

This shows how the heuristics work together in actual search.

8. Worked Understanding Using Map Coloring

Suppose several regions are already assigned colors, and the current remaining possibilities are:

- `WA: {red}`
- `NT: {green, blue}`
- `SA: {green, blue}`
- `Q: {red, blue}`

MRV chooses `WA` first because it has only one legal value left.

If later `NT` and `SA` both have two remaining values, then degree heuristic chooses the one connected to more unassigned neighbors.

After choosing the variable, LCV prefers the color that leaves the neighboring regions with the most flexibility.

> **Things to Think About:**
>
> - Why does MRV help even if it seems to focus on the “hardest” variable first?
> - Why is the degree heuristic only a tie-breaker and not the main rule?
> - Why might a locally attractive value be globally bad if it blocks too many future options?

9. Why These Heuristics Work Conceptually

The heuristics are all based on one major idea: **do not delay important information**.

- MRV brings likely failure points forward.

- Degree heuristic chooses variables with the largest impact on the remaining search.
- LCV avoids unnecessary damage to future choices.

Together, they reduce wasted exploration.

They do not guarantee polynomial-time solving, but in practice they can make an enormous difference.

> **Common Confusion:**
>
> Students sometimes think heuristics make the search “more correct.” That is wrong. The search was already correct. Heuristics make it **more efficient**, not more correct.

10. Limits of Heuristics

Heuristics improve search order, but they do not solve every difficulty.

- A hard CSP may still require large search.
- A heuristic can guide well in one problem and less well in another.
- Heuristics do not replace consistency checking or constraint propagation.

So heuristics are powerful, but they are only one layer of improvement.

11. Relationship to the Next Topic

Even with good heuristics, backtracking may still go down branches that are already doomed but not yet visibly impossible. The next improvement is to detect trouble earlier through methods such as forward checking and constraint propagation.

> **Bridge Forward:**
>
> Heuristics choose better branches. Constraint propagation will reduce bad branches before they grow.

12. Things to Think About

- Why is it often smart to choose the most constrained variable first rather than avoiding it?
- How does LCV differ from choosing the easiest-looking value?
- Why can a good value ordering reduce backtracking later?
- Can heuristics change the worst-case complexity of CSP solving, or do they mainly improve practical performance?

13. Quick Recap

- Heuristics improve backtracking by choosing variables and values more intelligently.
- MRV chooses the variable with the fewest remaining legal values.
- Degree heuristic breaks MRV ties by choosing the variable that constrains others the most.
- LCV chooses the value that rules out the fewest future options.
- These heuristics improve efficiency, not correctness.
- They work best as part of a smarter overall CSP-solving strategy.

14. Part 4 Revision Lines

> - **MRV:** choose the most constrained variable.
> - **Degree:** if tied, choose the variable that constrains the rest the most.
> - **LCV:** choose the value that leaves the most room for the future.

End of Part 4